1 Lesson 29-10-26

We hit the ground running from the previous lesson, continuing to focus on real time scheduling algorithms for real time systems.

1.1 Mixed job sets

We understood that real systems mix both kinds of work:

- **Periodic tasks**: time-driven, critical control loops with hard deadlines;
- **Aperiodic jobs**: event-driven (a button, a message, a fault); hard, soft or no deadline; arrival time not known in advance.

For mixed sets, let's forget the fact that aperiodic jobs have deadlines and assume that aperiodic jobs are served first come, first served (**FCFS**) among themselves (same arrival: lowest identifier). For periodic tasks let's stick to **RM** (7.1.1).

Three ways to serve them: in **background**, with a **polling server**, and with a **deferrable server**.

1.1.1 Background scheduling

In **background** scheduling the periodic tasks run with *RM*, an aperiodic job runs only when no periodic job is ready. This is the simplest to implement (requires basically no implementation) and focuses on periodic jobs as first-class.

We keep two ready queues: periodic (RM priority) and aperiodic (FCFS). The aperiodic queue is served only when the periodic one is empty. A periodic release preempts the aperiodic job at once (focus on periodic jobs). The periodic schedule is exactly the RM schedule without the aperiodic jobs: we only use the free slots left by RM if possible. This means the aperiodic response is the time that is left: it depends on the periodic load.

The problem of background scheduling is this last point: we know that in fact aperiodic tasks have deadlines, and with this approach we ignore them. This means that if the periodic task set utilizes the CPU fully (or almost fully) we might get serious aging problems of the aperiodic tasks, if not not executing them at all.

1.1.2 Polling server

A server is a periodic task created to run aperiodic jobs. It has a period (like any periodic task) T_s and a budget (capacity) C_s: the time it may use in each period. Its utilisation is $U_s = \frac{C_s}{T_s}$.

The **polling server** is scheduled with **RM** like any other periodic task: its priority comes from T_s. If T_s equals the period of a task, both have the same RM priority: the task runs first.

At each period start the budget si set to C_s. It never accumulates. The server runs only when it has budget and an aperiodic job is pending; each slot it runs costs 1 unit. Now:

- If no job is pending at its activation (the period start): the budget is lost (0) until the next period;
- If the queue becomes empty while it has budget left: the rest of the budget is lost;

- A job that arrives during a period is served in that period only if the budget is still there (a job was pending at the activation and the queue has not emptied yet); otherwise it waits for the next period (because the budget is lost).

There's a feasibility bound for the polling server:

$$P = \prod_{i}^{n}(U_i + 1) \leq \frac{2}{U_s + 1}$$

Where U_s refers to the server utilization given maximum capacity C_s:

$$U_s = \frac{C_s}{T_s}, \quad U_i = \frac{C_i}{T_i}$$

This bound is only *sufficient*, necessary (see 7.1.2 or the Liu-Layland notes). In fact, this is basically Liu-Layland including the server task at maximum capacity:

$$\prod_{i}^{n}(U_i + 1) \cdot (U_s + 1) \leq 2$$

which is also valid.

Proof
We can prove that the Liu–Layland utilization bound given in 7.12 or in the notes implies the product condition. The two conditions are not equivalent in general; the utilization bound is a sufficient condition for the product inequality:

$$\sum_{i}^{n} U_i \leq n\left(2^{\frac{1}{n}} - 1\right) \implies \prod_{i}^{n}(U_i + 1) \leq 2$$

This proof seems to come easy using the AM-GM inequality (or *Jensen*'s inequality if you fuck around long enough). Since we seemed to have fucked around long enough, in this proof we try to cover both bases.

First we start from the fact that:

$$\frac{1}{n}\sum_{i}^{n}\log\left(U_i + 1\right) \leq \log\left(\frac{1}{n}\sum_{i}\left(U_i + 1\right)\right)$$

This comes from observing that $\log(x)$ is a concave function. Because of this concavity, the the average of the function's values is no greater than the function evaluated at the average. Really, this statement is just Jensen's inequality (which generalizes to any function f that is concave).

What we can do is take exponents on both sides:

$$e^{\frac{1}{n}\sum_{i}^{n}\log(U_i+1)} \leq \frac{1}{n}\sum_{i}^{n}\left(U_i + 1\right)$$

$$\left(e^{\sum_{i}^{n}\log(U_i+1)}\right)^{\frac{1}{n}} \leq \frac{1}{n}\sum_{i}^{n}\left(U_i + 1\right)$$

$$\left(\prod_{i}^{n}\left(U_i + 1\right)\right)^{\frac{1}{n}} \leq \frac{1}{n}\sum_{i}^{n}\left(U_i + 1\right)$$

The last step is simply noticing at the LHS that:

$$\begin{aligned} e^{\sum_{i=1}^{n} \log(U_i+1)} &= e^{\log(U_1+1)+\log(U_2+1)+\cdots+\log(U_n+1)} \\ &= e^{\log(U_1+1)} e^{\log(U_2+1)} \cdots e^{\log(U_n+1)} \\ &= (U_1+1)(U_2+1)\cdots(U_n+1) \\ &= \prod_{i=1}^{n}(U_i+1) \end{aligned}$$

At this point, what we proved is basically the AM-GM inequality.

We can now take the LHS of the original equivalence:

$$\begin{aligned} \sum_{i}^{n} U_i &\leq n\left(2^{\frac{1}{n}}-1\right) \\ \frac{1}{n}\sum_{i}^{n} U_i &\leq 2^{\frac{1}{n}}-1 \\ \sum_{i}^{n} \frac{U_i}{n}+1 &\leq 2^{\frac{1}{n}} \\ \sum_{i}^{n} \frac{U_i}{n}+\sum_{i}^{n}\frac{1}{n} &\leq 2^{\frac{1}{n}} \\ \frac{1}{n}\sum_{i}^{n}(U_i+1) &\leq 2^{\frac{1}{n}} \end{aligned}$$

The LHS of this is the RHS of the inequality we proved earlier. If this is true, then it's also true that:

$$\begin{aligned} \left(\prod_{i}^{n}(U_i+1)\right)^{\frac{1}{n}} &\leq 2^{\frac{1}{n}} \\ \prod_{i}^{n}(U_i+1) &\leq 2 \end{aligned}$$

The last step was the same as applying the AM-GM inequality, thus the whole ordeal.

Sometimes this multiplicatory bound is called the *hyperbolic bound* (e.g. Wikipedia), which is a tighter sufficient condition for schedulability than the one originally presented by Liu and Layland.

"*Tighter*" here means *less conservative*: the Liu-Layland proof (again, read the notes) assumes the worst case scenario (called the *critical instant*) where all tasks are scheduled at the same instant. The multiplicative bound doesn't consider just this particular condition, so where the Liu–Layland bound depends only on the total utilization, it retains information about the individual task utilizations. Consequently, the hyperbolic test can distinguish between task sets with the same total utilization but different distributions of utilization across tasks.

The implication means this: all sets schedulable according to the Liu-Layland bound are schedulable according to the hyperbolic bound. Some other sets schedulable according to the hyperbolic bound might be missed by the Liu-Layland bound.

□

This bound is useful for determining the polling server *capacity*, that is **sizing** the polling server. We solve the test for U_s: the largest server utilisation is:

$$U_{s\max} = (2 - P)/P$$

Where P is the product of other tasks. As a rule of thumb:

- T_s is set equal to the shortest period: the server gets the highest priority (equal to that task, which runs first);
- $C_s = U_s \cdot T_s$ with $U_s \leq U_{s\max}$; at the exam, the largest integer C_s.

1.1.3 Deferrable server

The polling server wastes its budget when no job is pending at the right moment, so a job that arrives just after waits a whole period. The **deferrable server** keeps the budget.

We assume the same parameters C_s, T_s, same RM priority from T_s (it loses a tie with a task of the same period). The difference is that it keeps its budget until the end of its period, even when no job is pending. It serves an aperiodic job at any time while budget is left: an arriving job can preempt a lower-priority task at once. At each period start the budget is set back to C_s: not added to what is left.

This is way better at servicing aperiodic tasks. The price is **back-to-back** execution: the deferrable server can use its budget at the end of one period and again at the start of the next. Up to $2C_s$ can be used in a row, without servicing any other task.

Because of this last consideration, we know that the deferrable server is not a plain periodic task: back-to-back it can run $2C_s$ in a row, so it needs its own test (sufficient only):

$$P = \prod_i^n (U_i + 1) \leq \frac{U_s + 2}{2U_s + 1}$$

This bound is also useful for sizing the polling server:

$$U_{s\max} = \frac{(2 - P)}{2P - 1}$$

Same as above, we have a rule of thumb:

- T_s is set equal to the shortest period: the server gets the highest priority (equal to that task, which runs first);
- $C_s = U_s \cdot T_s$ with $U_s \leq U_{s\max}$; at the exam, the largest integer C_s.

1.2 Sources of unpredictability

Every test of this week assumes that C is a true upper bound (the *Worst Case Execution Time*, **WCET**) and that nothing else takes the CPU. A real system must be predictable: its timing can be bounded in advance. Sources of unpredictability around the CPU: DMA steals bus cycles; cache hit or miss varies execution time; interrupts cause unbounded preemption; system calls run in non-preemptive kernel sections; memory management adds paging and allocation delays. All make WCET and response time hard to bound.

- **DMA**: the DMA controller and the CPU share the bus. In burst mode the CPU waits for a whole block; in cycle stealing it waits one word at a time. Fix: a time-sliced bus (fixed slots for CPU and DMA): slower, but the task time no longer depends on the transfer;

- **Cache**: a hit is fast, a miss is slow. The average improves, the worst case is hard to compute; each preemption destroys locality and adds misses (one measure on a PowerPC: WCET +33%). Fix: disable or lock the cache, or a larger, safe WCET;
- **Interrupts**: an ISR preempts any task, and the number of interrupts in a window is hard to bound. Fix: a very short ISR that only wakes a task that does the work, scheduled with its own priority; or treat the device as aperiodic jobs served by a server, as in this unit;
- **System calls**: a non-preemptible kernel section delays even the highest-priority task. Their time must be known and bounded, and the kernel made preemptible;
- **Memory management**: paging and dynamic allocation take a time that depends on the state of memory. Real-time code allocates at start-up, avoids recursion and unbounded loops.